

Module 2: Transport Layer and Network Layer



Module 2: Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
 - select()
 - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using ip

Transport-Layer Protocols — Introduction

- Transport-layer protocols are created by:
 - Combining different transport-layer services
- These services include:
 - Flow Control
 - Error Control
 - Congestion Control
 - Connection Management
- Different combinations produce:
 - Different transport-layer protocols



Transport-Layer Protocols — Protocol Development Approach

- To understand transport protocols:
 - We begin with the simplest protocol
 - Complexity is added gradually
- This approach helps in understanding:
 - Basic protocol behavior
 - Advanced protocol mechanisms
- TCP/IP transport protocols are:
 - Modifications or combinations of these basic protocols

Transport-Layer Protocols — Purpose of Studying General Protocols

- General transport protocols are studied to:
 - Build a strong foundation
 - Understand complex protocols later
- These concepts help in understanding:
 - TCP
 - UDP
 - Reliable communication mechanisms

Key Idea

Simple protocols provide the basis for advanced transport-layer protocols.

Transport-Layer Protocols — Unidirectional Communication

- Initially, protocols are discussed as:

Simplex (Unidirectional) Protocols

Data moves only in one direction

- Characteristics:
 - One sender
 - One receiver
 - Data flows in only one direction
- Purpose:
 - Simplifies protocol understanding

Transport-Layer Protocols — Bidirectional Communication

- Later, protocols are extended to:

Full Duplex (Bidirectional) Protocols

Data moves in both directions

- Characteristics:

- Both sides can send data
- Both sides can receive data
- Simultaneous communication is possible

- Example:

- TCP communication

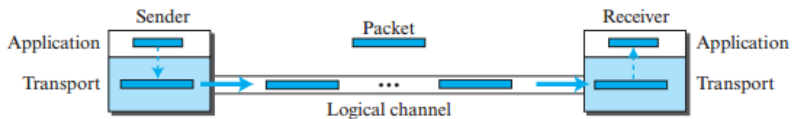


Simple Protocol — Introduction

- The Simple Protocol is:
 - A connectionless protocol
- Features:
 - No Flow Control
 - No Error Control
- Assumption:
 - Receiver can immediately process incoming packets
 - Receiver is never overwhelmed

Simple Protocols

Figure 3.17 *Simple protocol*



Simple Protocol — Operation at Sender and Receiver

- Sender Transport Layer:

- ① Receives message from Application Layer
- ② Encapsulates message into a packet
- ③ Sends packet to receiver

- Receiver Transport Layer:

- ① Receives packet from Network Layer
- ② Decapsulates the packet
- ③ Delivers message to Application Layer

Key Point

Transport layers provide transmission services to application layers.

Simple Protocol — Finite State Machines (FSMs)

- Both sender and receiver use:

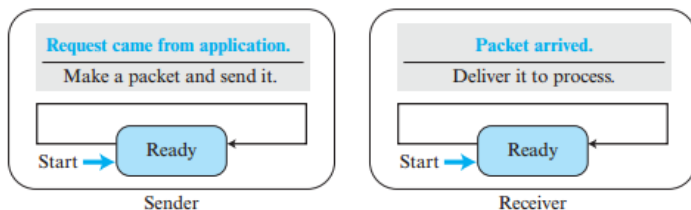
Finite State Machines (FSMs)

Machines that remain in a defined state until an event occurs

- Each FSM contains only one state:
 - Ready State
- Sender waits for:
 - Message from Application Layer
- Receiver waits for:
 - Incoming packet

FSMs for the simple protocol

Figure 3.18 *FSMs for the simple protocol*



Simple Protocol — Sender FSM Operation

- Sender machine remains in:
 - Ready State
- Event:
 - Application Layer sends a message
- Sender actions:
 - 1 Encapsulate message into packet
 - 2 Send packet to receiver
- Sender then returns to:
 - Ready State

Simple Protocol — Receiver FSM Operation

- Receiver machine remains in:
 - Ready State
- Event:
 - Packet arrives from sender
- Receiver actions:
 - 1 Decapsulate message from packet
 - 2 Deliver message to Application Layer
- Receiver then returns to:
 - Ready State

Simple Protocol — Communication Example

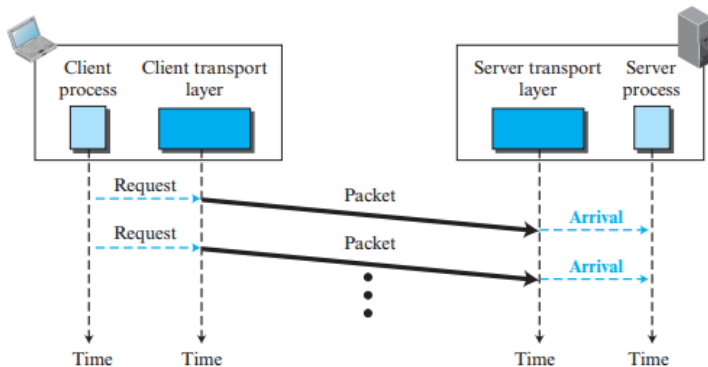
- Communication is very simple:
 - Sender continuously sends packets
 - Receiver accepts packets immediately
- Characteristics:
 - No acknowledgments
 - No retransmissions
 - No packet numbering
 - No coordination between sender and receiver

Important Point

UDP is a slight modification of this simple protocol.

Flow diagram for Example 3.3

Figure 3.19 *Flow diagram for Example 3.3*



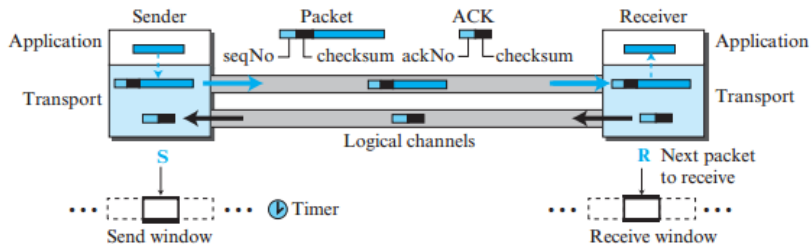
Stop-and-Wait Protocol — Introduction

- Stop-and-Wait is:
 - A connection-oriented protocol
- Provides:
 - Flow Control
 - Error Control
- Both sender and receiver use:
 - Sliding Window of size 1
- Sender:
 - Sends one packet at a time
 - Waits for acknowledgment before sending next packet



Stop and Wait Protocol

Figure 3.20 Stop-and-Wait protocol



Stop-and-Wait Protocol — Error Detection

- Each packet contains:
 - Checksum field
- Receiver checks the checksum:
 - Correct checksum → Packet accepted
 - Incorrect checksum → Packet discarded
- Receiver does not send response for corrupted packets

Important Point

Silence from receiver indicates packet loss or corruption.

Stop-and-Wait Protocol — Timers and Retransmission

- Sender starts a timer after sending a packet
- If ACK arrives before timeout:
 - Timer stops
 - Sender transmits next packet
- If timeout occurs:
 - Sender assumes packet/ACK was lost
 - Packet is retransmitted
- Sender keeps a copy of packet until ACK arrives

Stop-and-Wait Protocol — Sequence Numbers

- Sequence numbers prevent:
 - Duplicate packets
- Packet header contains:
 - Sequence Number field
- Only two sequence numbers are needed:

0 and 1

- Sequence numbers follow:

Modulo-2 Arithmetic

0, 1, 0, 1, 0, ...

Stop-and-Wait Protocol — Why Only 0 and 1?

- Assume packet x is sent
- Three possible situations:
 - 1 Packet arrives correctly and ACK arrives
 - 2 Packet is lost/corrupted
 - 3 ACK is lost/corrupted
- Receiver distinguishes:
 - New packet
 - Duplicate packet
- Therefore:
 - Only sequence numbers 0 and 1 are sufficient

Stop-and-Wait Protocol — Acknowledgment Numbers

- ACK number announces:

Next Expected Packet

Receiver informs sender about the next expected sequence number

- Example:

- Packet 0 received → ACK 1 sent
- Packet 1 received → ACK 0 sent

- ACK numbers also use:

Modulo-2 arithmetic

Stop-and-Wait Protocol — Sender Variables

- Sender uses control variable:

S

- Receiver uses control variable:

R

- S :

- Points to sender window slot

- R :

- Points to receiver window slot

Stop-and-Wait Protocol — Sender FSM

- Sender has two states:

- ① Ready State
- ② Blocking State

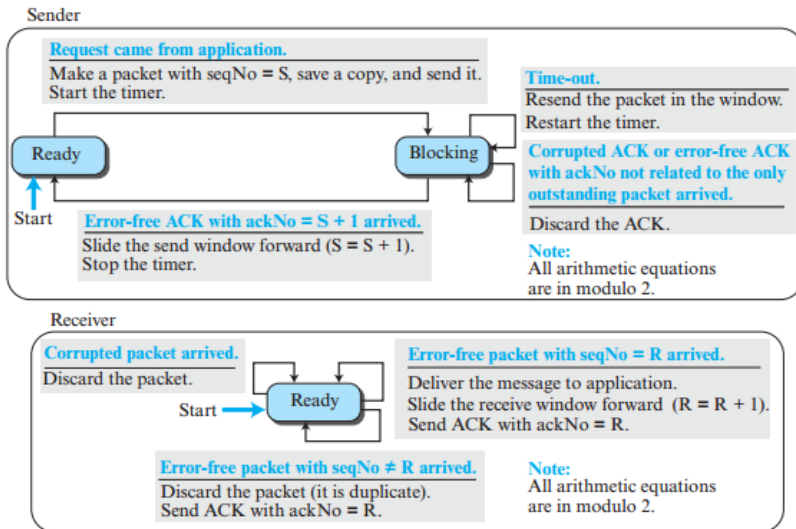
- Ready State:

- Waits for message from application
- Creates packet with sequence number S
- Sends packet and starts timer
- Moves to Blocking State



FSM for the Stop-and-Wait protocol

Figure 3.21 FSM for the Stop-and-Wait protocol



Stop-and-Wait Protocol — Blocking State

- In Blocking State, three events may occur:

- 1 Correct ACK arrives:

- Timer stops
- Window slides
- Sender returns to Ready State

- 2 Corrupted/Wrong ACK arrives:

- ACK discarded

- 3 Timeout occurs:

- Outstanding packet retransmitted
- Timer restarted

Stop-and-Wait Protocol — Receiver FSM

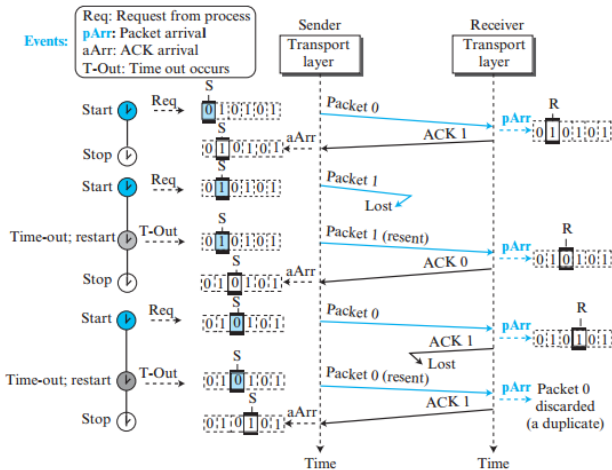
- Receiver always remains in:
 - Ready State
- Possible events:
 - 1 Correct packet with expected seqNo arrives:
 - Deliver data
 - Slide window
 - Send ACK
 - 2 Duplicate packet arrives:
 - Packet discarded
 - ACK resent
 - 3 Corrupted packet arrives:
 - Packet discarded

Stop-and-Wait Protocol — Example 3.4

- Packet 0:
 - Sent successfully
 - ACK received
- Packet 1:
 - Lost during transmission
 - Resent after timeout
 - ACK received
- Packet 0:
 - ACK lost
 - Sender retransmits packet
 - Receiver detects duplicate

Example

Figure 3.22 Flow diagram for Example 3.4



Stop-and-Wait Protocol — Efficiency

- Stop-and-Wait is inefficient for:
 - High bandwidth channels
 - Long-delay networks

- Important term:

Bandwidth-Delay Product

Number of bits that can fill the communication channel

- Channel behaves like:
 - A pipe carrying bits

Stop-and-Wait Protocol — Example 3.5

- Given:

- Bandwidth = 1 *Mbps*
- Round-trip time = 20 *ms*
- Packet size = 1000 *bits*

- Bandwidth-delay product:

$$(1 \times 10^6)(20 \times 10^{-3}) = 20,000 \text{ bits}$$

- Utilization:

$$\frac{1000}{20,000} = 5\%$$

Observation

Stop-and-Wait wastes network capacity.

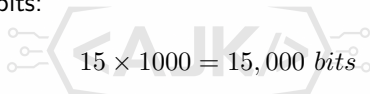
Stop-and-Wait Protocol — Example 3.6

- Assume protocol sends:

15 packets

before waiting for ACKs

- Total transmitted bits:


$$15 \times 1000 = 15,000 \text{ bits}$$

- Utilization:

$$\frac{15,000}{20,000} = 75\%$$

- Efficiency increases significantly

Stop-and-Wait Protocol — Pipelining

- Pipelining:

Definition

Starting a new transmission before the previous one finishes

- Stop-and-Wait does NOT support pipelining because:
 - Sender waits for ACK before sending next packet
- Future protocols allow:
 - Multiple packets in transit simultaneously
- Pipelining improves:
 - Transmission efficiency

Go-Back-N Protocol (GBN) — Introduction

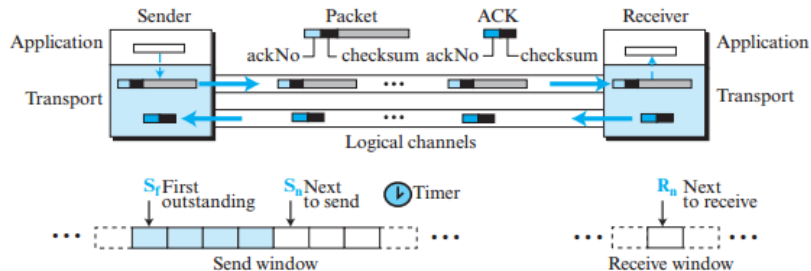
- Stop-and-Wait suffers from poor utilization
- To improve efficiency:
 - Multiple packets must be in transit simultaneously
- Go-Back-N (GBN):
 - Allows several outstanding packets
 - Improves channel utilization
 - Uses flow and error control

- Receiver can buffer only:

1 packet

Go-Back-N protocol

Figure 3.23 Go-Back-N protocol



Go-Back-N Protocol (GBN) — Basic Characteristics

- Sender:
 - Can send multiple packets before receiving ACKs
- Receiver:
 - Accepts only the expected packet
 - Discards out-of-order packets
- Sender stores copies of:
 - All outstanding packets
- Multiple packets and ACKs can exist in the channel simultaneously

Go-Back-N Protocol (GBN) — Sequence Numbers

- Sequence numbers use:

$$0 \text{ to } 2^m - 1$$

- Arithmetic follows:

Modulo- 2^m Arithmetic

Sequence numbers wrap around after reaching the maximum value.

- Example:

$$m = 3 \Rightarrow 0, 1, 2, 3, 4, 5, 6, 7$$

Go-Back-N Protocol (GBN) — Acknowledgment Numbers

- ACKs are:

Cumulative Acknowledgments

- ACK number indicates:

- Next expected packet

- Example:

$$ACK = 7$$

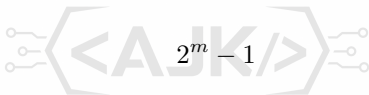
- Meaning:

- Packets 0 through 6 received successfully
- Receiver expects packet 7

Go-Back-N Protocol (GBN) — Send Window

- Send Window:
 - Imaginary box containing packets that can be sent

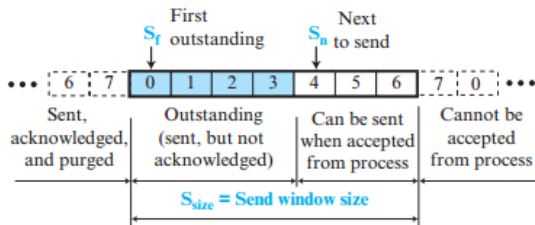
- Maximum size:

A diagram showing a large hexagonal shape with circuit-like lines extending from its left and right sides. Inside the hexagon, the mathematical expression $2^m - 1$ is centered. A large, faint watermark 'ARSHA JK' is visible across the center of the diagram.
$$2^m - 1$$

- Window contains:
 - Sent but unacknowledged packets
 - Packets ready to be sent

Send Window

Figure 3.24 *Send window for Go-Back-N*



Go-Back-N Protocol (GBN) — Four Regions of Send Window

- ① Acknowledged packets
 - Already received successfully
- ② Outstanding packets
 - Sent but not yet acknowledged
- ③ Ready-to-send packets
 - Allowed to be transmitted
- ④ Future packets
 - Cannot be sent until window slides

Go-Back-N Protocol (GBN) — Send Window Variables

- Three variables define the send window:

Sf First outstanding packet
Sn Next packet to send
Ssize Window size

- Window is an abstraction implemented using these variables

Go-Back-N Protocol (GBN) — Sliding of Send Window

- Window slides when:
 - Valid ACK arrives

- Example:



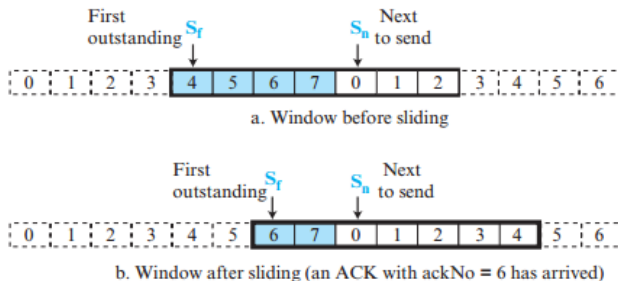
- Means:
 - Receiver expects packet 6
 - Packets before 6 are acknowledged

- Sender updates:

$$Sf = ACK$$

Sliding the Send Window

Figure 3.25 *Sliding the send window*



Go-Back-N Protocol (GBN) — Receive Window

- Receive window size:

1

- Receiver accepts:

- Only one expected packet

- Out-of-order packets:

- Discarded immediately

- Receiver variable:

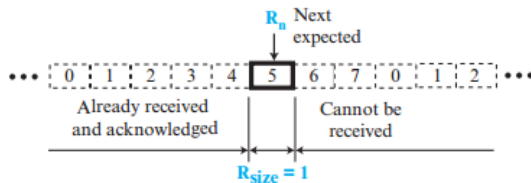
R_n

- Represents:

- Next expected packet

Receive Window

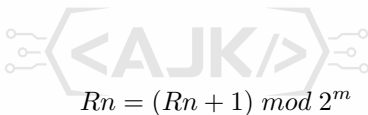
Figure 3.26 *Receive window for Go-Back-N*



Go-Back-N Protocol (GBN) — Receive Window Sliding

- When packet Rn arrives correctly:

- 1 Deliver data
- 2 Send ACK
- 3 Slide window


$$Rn = (Rn + 1) \bmod 2^m$$

- Window slides:
 - One position at a time

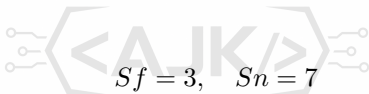
Go-Back-N Protocol (GBN) — Timer Mechanism

- Multiple outstanding packets exist
- Only one timer is used
- Timer belongs to:
 - Oldest outstanding packet
- Reason:
 - First outstanding packet expires first

Go-Back-N Protocol (GBN) — Why Called Go-Back-N?

- If timeout occurs:
 - Sender returns to first unacknowledged packet
 - Resends all outstanding packets

- Example:


$$Sf = 3, \quad Sn = 7$$

- Outstanding packets:

3, 4, 5, 6

- Sender retransmits:

3, 4, 5, 6

Go-Back-N Protocol (GBN) — Sender FSM

- Sender states:

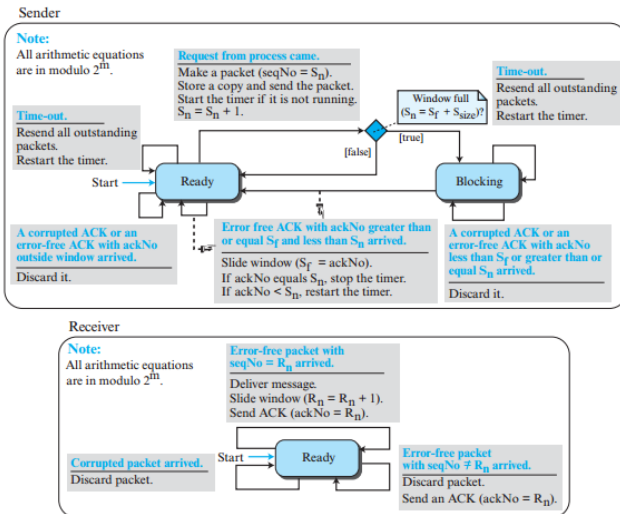
- 1 Ready State
- 2 Blocking State

- Initially:


$$Sf = Sn = 0$$

- Sender alternates between these states depending on window availability

Figure 3.27 FSMs for the Go-Back-N protocol



Go-Back-N Protocol (GBN) — Ready State

- Possible events:
 - 1 New packet arrives from application
 - 2 Valid ACK arrives
 - 3 Invalid ACK arrives
 - 4 Timeout occurs
- If window becomes full:
 - Sender enters Blocking State

Go-Back-N Protocol (GBN) — Blocking State

- Window is full
- Sender cannot accept new packets
- Possible events:
 - 1 Valid ACK
 - 2 Invalid ACK
 - 3 Timeout
- Valid ACK causes:
 - Window slide
 - Return to Ready State



Go-Back-N Protocol (GBN) — Receiver FSM

- Receiver always remains in:

Ready State

- Events:
 - Expected packet arrives
 - Out-of-order packet arrives
 - Corrupted packet arrives

Go-Back-N Protocol (GBN) — Receiver Actions

- Correct expected packet:
 - Deliver data
 - Slide window
 - Send ACK
- Out-of-order packet:
 - Discard packet
 - Send ACK for expected packet
- Corrupted packet:
 - Discard packet



Go-Back-N Protocol (GBN) — Send Window Size Restriction

- Send window size must satisfy:

$$Ssize < 2^m$$

- Therefore:



$$Ssize \leq 2^m - 1$$

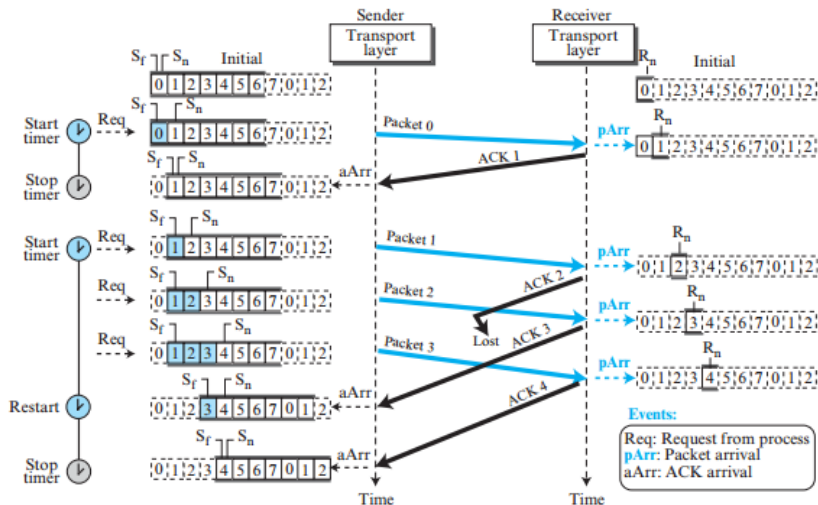
- Prevents ambiguity between:
 - New packets
 - Duplicate packets

Go-Back-N Protocol (GBN) — Example 3.7

- Forward channel:
 - Reliable
- Reverse channel:
 - ACK delays and losses occur
- ACK 2 is lost
- ACK 3 acts as:
 - ACK 2 + ACK 3
- Demonstrates cumulative acknowledgment

Example 3.7

Figure 3.29 Flow diagram for Example 3.7



Go-Back-N Protocol (GBN) — Example 3.8

- Packet 1 is lost
- Receiver receives:

2,3

- Receiver discards them because:
 - Packet 1 is expected

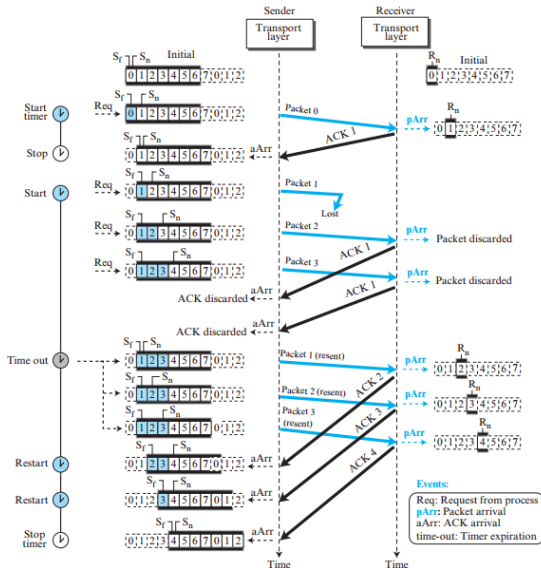
- Receiver repeatedly sends:

ACK1

- Timeout causes sender to retransmit:

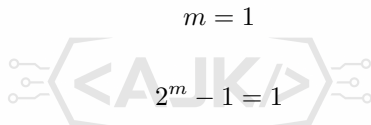
1, 2, 3

Figure 3.30 *Flow diagram for Example 3.8*



Go-Back-N Protocol (GBN) — Comparison with Stop-and-Wait

- Stop-and-Wait is a special case of GBN
- Parameters:

$$m = 1$$

$$2^m - 1 = 1$$

- Therefore:
 - Only two sequence numbers
 - Window size = 1
 - One outstanding packet

Conclusion

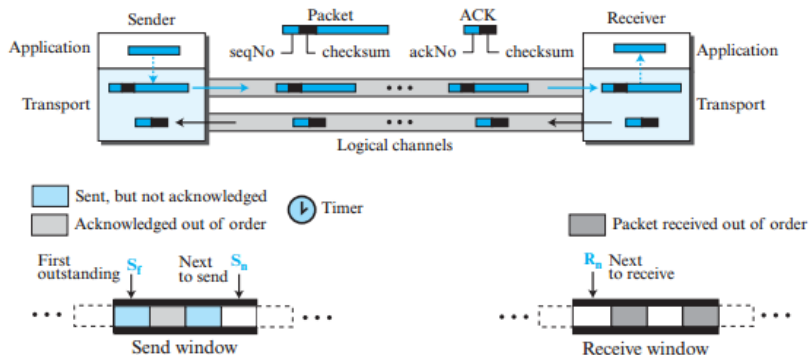
Stop-and-Wait = Go-Back-N with window size 1.

Selective-Repeat Protocol — Introduction

- Go-Back-N has a limitation:
 - Resends all outstanding packets when one packet is lost
- This causes:
 - Unnecessary retransmissions
 - Increased network congestion
- Selective-Repeat (SR):
 - Retransmits only lost packets
 - Improves efficiency
 - Reduces congestion

Outline of Selective repeat

Figure 3.31 *Outline of Selective-Repeat*



Selective-Repeat Protocol — Key Idea

- Sender:
 - Keeps copies of outstanding packets
- Receiver:
 - Accepts out-of-order packets
 - Buffers them temporarily
- Only missing packets are retransmitted

Advantage

Better performance than Go-Back-N when packet losses occur.

Selective-Repeat Protocol — Send Window

- SR uses a send window
- Maximum window size:

$$2^m - 1$$

- Example:

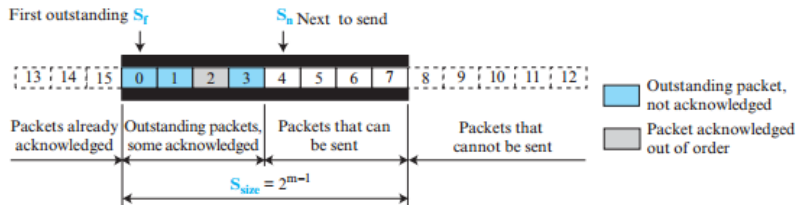


Window Size = 8

- Compare:
 - GBN: Window size = 15
 - SR: Window size = 8

Send window of Selective repeat

Figure 3.32 *Send window for Selective-Repeat protocol*



Selective-Repeat Protocol — Receive Window

- Receive window size:

$$\textit{Receive Window} = \textit{Send Window}$$

- Receiver can:

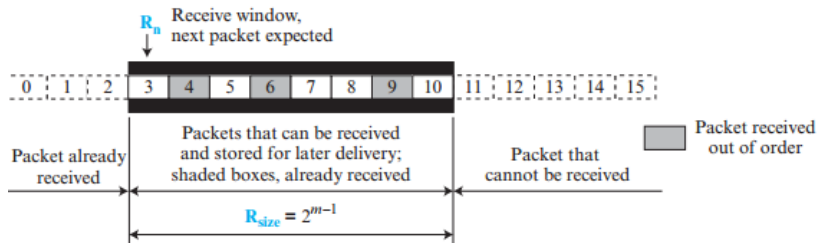
- Accept out-of-order packets
- Store them in the buffer

- Unlike GBN:

- Out-of-order packets are not discarded

Receive window of Selective repeat

Figure 3.33 *Receive window for Selective-Repeat protocol*



Selective-Repeat Protocol — Packet Delivery Rule

- Packets may arrive out of order
- Receiver stores packets until:
 - Missing packets arrive
- Packets are delivered only when:
 - 1 Consecutive packets are available
 - 2 Sequence starts from beginning of receive window

Important

Reliable transport layers never deliver packets out of order.

Selective-Repeat Protocol — Timer Mechanism

- Theoretical implementation:
 - One timer per outstanding packet
- When a timer expires:
 - Only the corresponding packet is retransmitted
- Difference from GBN:

GBN	SR
Retransmit All	Retransmit Selectively

Selective-Repeat Protocol — Acknowledgments

- ACK meaning is different from GBN
- In GBN:

ACK = Next Expected Packet



- In SR:

ACK = Packet Successfully Received

Definition

ACK number identifies one specific packet received correctly.

Selective-Repeat Protocol — Example 3.9

- Sender transmits:

0, 1, 2, 3, 4, 5

- Receiver sends:



GBN Interpretation

- Packets 0,1,2 received
- Receiver expects packet 3

SR Interpretation

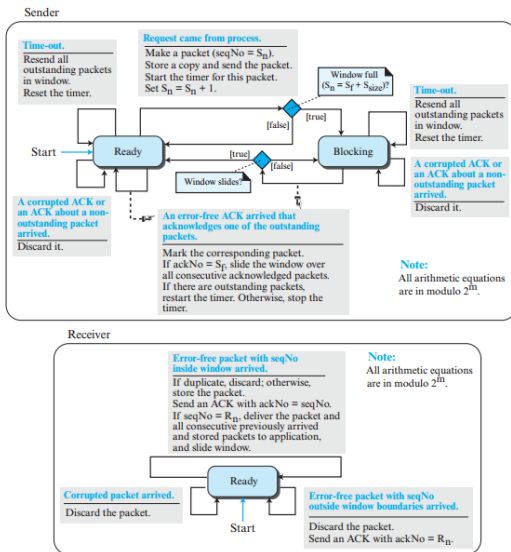
- Packet 3 received correctly
- No information about other packets

Selective-Repeat Protocol — Sender FSM

- Sender states:
 - 1 Ready State
 - 2 Blocking State
- Events handled:
 - Packet request
 - ACK arrival
 - Invalid ACK
 - Timeout
- ACK marks corresponding packet as acknowledged

FSM of Selective repeat

Figure 3.34 FSMs for SR protocol



Selective-Repeat Protocol — Sender Window Sliding

- ACK received for packet:

$$ackNo = Sf$$

- Window slides until:
 - First unacknowledged packet is found
- Consecutive acknowledged packets leave the window
- Timer:
 - Restarted if packets remain
 - Stopped if all packets acknowledged

Selective-Repeat Protocol — Receiver FSM

- Receiver remains in:

Ready State

- Possible events:
 - 1 Packet inside window
 - 2 Packet outside window
 - 3 Corrupted packet

Selective-Repeat Protocol — Receiver Actions

- Packet inside window:
 - Store packet
 - Send ACK
 - Deliver if consecutive sequence exists
- Packet outside window:
 - Discard packet
 - Send ACK
- Corrupted packet:
 - Discard packet

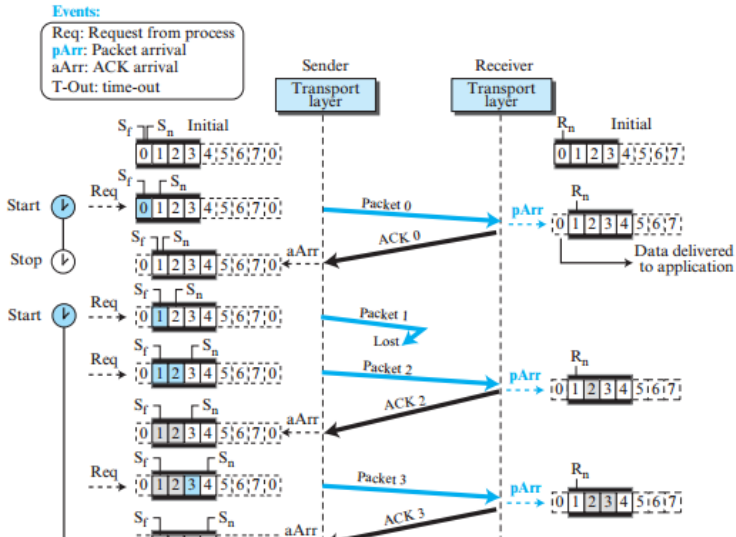
Selective-Repeat Protocol — Example 3.10

- Packet 1 is lost
- Packets 2 and 3:
 - Arrive out of order
 - Stored in buffer
 - Acknowledged
- Timer expires
- Only packet 1 is retransmitted
- After packet 1 arrives:
 - Packets 1, 2, and 3 delivered in order



Example

Figure 3.35 *Flow diagram for Example 3.10*



Selective-Repeat Protocol — Window Size Limitation

- Sender and receiver windows must satisfy:

$$\text{Window Size} \leq 2^m - 1$$

- Reason:

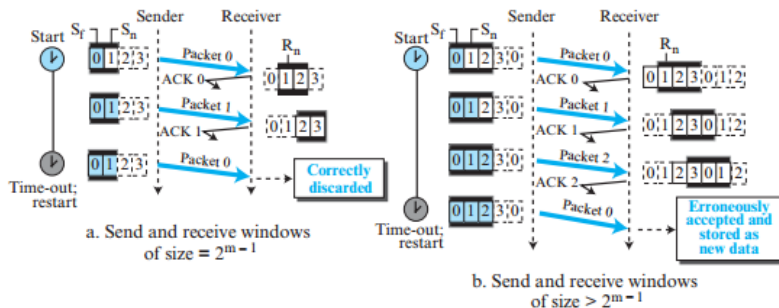
- Avoid confusion between
 - New packets
 - Duplicate packets

- Larger windows may cause:

- Duplicate packets to be accepted as new packets

window size

Figure 3.36 Selective-Repeat, window size



Selective-Repeat Protocol — GBN vs SR

Go-Back-N	Selective-Repeat
Receive window = 1	Receive window = Send window
Out-of-order packets discarded	Out-of-order packets buffered
Cumulative ACKs	Individual ACKs
Retransmit all outstanding packets	Retransmit only lost packets
Less efficient	More efficient
Simpler implementation	More complex implementation

Bidirectional Protocols: Piggybacking — Introduction

- Previous protocols were:
 - Unidirectional (Simplex)
- Data packets flowed:
 - In one direction
- Acknowledgments flowed:
 - In the opposite direction
- Real-world communication is usually:

Bidirectional (Full Duplex)

Data flows simultaneously in both directions.

Bidirectional Protocols: Piggybacking — Need for Piggybacking

- In bidirectional communication:
 - Client sends data to server
 - Server sends data to client
- Both sides must also send:
 - Acknowledgments
- Sending separate ACK packets causes:
 - Additional overhead
 - Lower efficiency

Bidirectional Protocols: Piggybacking — Concept

- Piggybacking improves efficiency by:
 - Combining data and acknowledgment in the same packet
- Example:
 - Packet from A to B carries:
 - 1 Data from A
 - 2 ACK for packets received from B
- Similarly:
 - Packet from B to A carries:
 - 1 Data from B
 - 2 ACK for packets received from A

Bidirectional Protocols: Piggybacking — Working Principle

$A \Rightarrow B$

Data + ACK

$B \Rightarrow A$

Data + ACK

- Every data packet can carry:
 - User data
 - Acknowledgment information
- Separate ACK packets are often unnecessary

Bidirectional Protocols: Piggybacking — Advantages

- Reduces network overhead
- Reduces number of packets transmitted
- Improves bandwidth utilization
- Improves transmission efficiency
- Reduces processing load on network devices

Bidirectional Protocols: Piggybacking — GBN with Piggybacking

- Go-Back-N can be implemented as:

Bidirectional GBN

Data and ACKs flow in both directions simultaneously.

- Client and Server each maintain:
 - ① Send Window
 - ② Receive Window
- ACK information is piggybacked onto outgoing data packets

Bidirectional Protocols: Piggybacking — Window Management

- Each host acts as both:
 - Sender
 - Receiver
- Therefore each host maintains:

- 1 Send Window
 - 2 Receive Window
- 

- Example:

$$Client : \begin{cases} Send Window \\ Receive Window \end{cases}$$

$$Server : \begin{cases} Send Window \\ Receive Window \end{cases}$$

Bidirectional Protocols: Piggybacking — Key Points

- Data transmission occurs in both directions
- ACK information is attached to outgoing data packets
- Improves protocol efficiency
- Commonly used in:
 - Go-Back-N
 - Selective Repeat
 - TCP

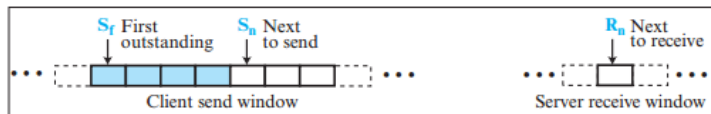
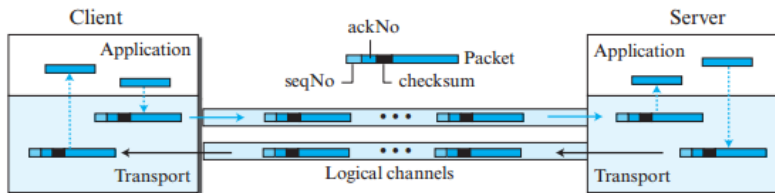


Summary

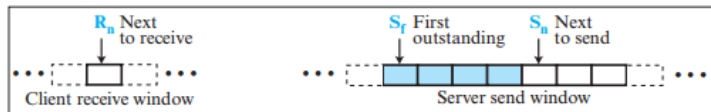
Piggybacking combines data and acknowledgments in a single packet to improve network efficiency.

Design of piggybacking in Go-Back-N

Figure 3.37 Design of piggybacking in Go-Back-N



Windows for communication from client to server

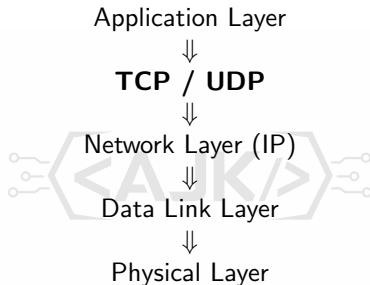


Windows for communication from server to client

Internet Transport-Layer Protocols — Introduction

- The Internet uses several transport-layer protocols
- Two major transport protocols are:
 - 1 UDP (User Datagram Protocol)
 - 2 TCP (Transmission Control Protocol)
- These protocols operate between:
 - Application Layer
 - Network Layer

Internet Transport-Layer Protocols — Position in TCP/IP Suite



- TCP and UDP act as intermediaries between:
 - Application programs
 - Network operations

- UDP stands for:

User Datagram Protocol

- Characteristics:

- Connectionless
- Unreliable
- Lightweight
- Fast



- Suitable when:

- Speed is more important than reliability

Internet Transport-Layer Protocols — Why Use UDP?

- UDP is preferred because of:
 - 1 Simplicity
 - 2 Low overhead
 - 3 High efficiency
- Error control, if needed, is handled by:
 - Application Layer
- Common applications:
 - DNS
 - Streaming
 - Online gaming
 - VoIP

- TCP stands for:

Transmission Control Protocol

- Characteristics:

- Connection-Oriented
- Reliable
- Ordered Delivery
- Error Controlled

- Suitable when:

- Reliability is essential

Internet Transport-Layer Protocols — Why Use TCP?

- TCP provides:
 - ➊ Reliable Data Delivery
 - ➋ Flow Control
 - ➌ Error Control
 - ➍ Congestion Control
- Common applications:
 - Web Browsing (HTTP/HTTPS)
 - Email
 - File Transfer
 - Remote Login



Internet Transport-Layer Protocols — UDP vs TCP

UDP	TCP
Connectionless	Connection-Oriented
Unreliable	Reliable
No flow control	Flow control available
No congestion control	Congestion control available
Lower overhead	Higher overhead
Faster	Slower but reliable

Internet Transport-Layer Protocols — Process-to-Process Communication

- A major responsibility of TCP and UDP is:

Process-to-Process Communication

- Achieved using:
 - Port Numbers
- IP Address identifies:
 - Host (Computer)
- Port Number identifies:
 - Process (Application)



Internet Transport-Layer Protocols — Summary

- Internet primarily uses:
 - 1 UDP
 - 2 TCP
- UDP:
 - Fast and connectionless
- TCP:
 - Reliable and connection-oriented
- Both protocols:
 - Enable process-to-process communication
 - Use port numbers for addressing applications

Position of transport-layer protocols in the TCP/IP protocol suite

Figure 3.38 *Position of transport-layer protocols in the TCP/IP protocol suite*

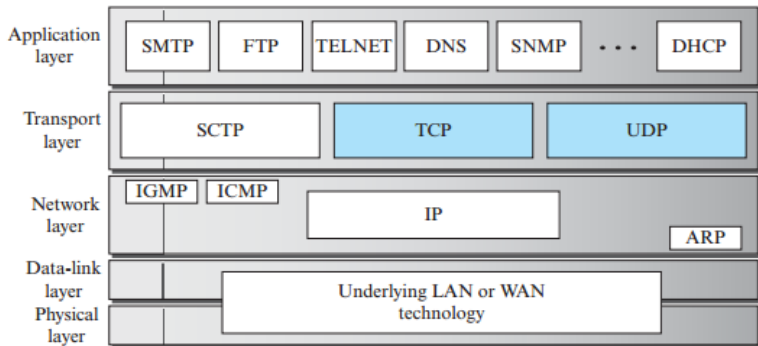


Table 3.1 *Some well-known ports used with UDP and TCP*

Port	Protocol	UDP	TCP	Description
7	Echo	√		Echoes back a received datagram
9	Discard	√		Discards any datagram that is received
11	Users	√	√	Active users
13	Daytime	√	√	Returns the date and the time
17	Quote	√	√	Returns a quote of the day
19	Chargen	√	√	Returns a string of characters
20, 21	FTP		√	File Transfer Protocol
23	TELNET		√	Terminal Network
25	SMTP		√	Simple Mail Transfer Protocol
53	DNS	√	√	Domain Name Service
67	DHCP	√	√	Dynamic Host Configuration Protocol
69	TFTP	√		Trivial File Transfer Protocol
80	HTTP		√	Hypertext Transfer Protocol
111	RPC	√	√	Remote Procedure Call
123	NTP	√	√	Network Time Protocol
161, 162	SNMP		√	Simple Network Management Protocol